المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵉⵏⵎⵍⵍⴰⵏ - ⴰⵎⵜ ⵎⵉⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

COMPTE RENDU

Travaux Pratiques — TP.01

Programmation Orientée Objet en Java

Étudiant	BANAN Fouad
Module	Génie Logiciel
Filière	Transformation Digitale (Individuelle)
Professeur	Pr. BE.ELBAGHAZAOUI
Année universitaire	2024–2025

Exercice 1.1 – Gestion d'animaux dans un Zoo

Partie 1 : Définitions théoriques

Question 1 — Encapsulation

L'encapsulation est un principe fondamental de la POO qui consiste à regrouper les données (attributs) et les traitements (méthodes) au sein d'une même classe, tout en contrôlant l'accès à ces données depuis l'extérieur grâce aux modificateurs de visibilité.

Dans la classe Animal :

- L'attribut nom est déclaré private → il est inaccessible directement depuis l'extérieur de la classe.
- L'accès à nom se fait uniquement via le getter public getNom(), qui constitue le seul point d'accès contrôlé.
- Aucun setter n'est fourni → l'attribut est en lecture seule après construction (immuabilité).

```java
abstract class Animal {
    private String nom;

    public Animal(String nom) { this.nom = nom; }

    public String getNom() { return nom; }

    public abstract void faireDuBruit();
}
```

Figure 1 — Classe abstraite Animal (encapsulation + abstraction)

Question 2 — Héritage

L'héritage est un mécanisme permettant à une classe enfant de réutiliser les attributs et méthodes d'une classe parent, tout en pouvant les spécialiser. En Java, il est matérialisé par le mot-clé extends.

Dans le code :

- Mammifere extends Animal → Mammifere hérite de getNom() et du constructeur via super().
- Oiseau extends Animal → même héritage, avec en plus un comportement propre aux oiseaux (voler()).

```java
class Mammifere extends Animal {

    public Mammifere(String nom) { super(nom); }

    @Override
    public void faireDuBruit() { System.out.println(getNom() + " grogne."); }
}
```

Figure 2 — Classe Mammifere (héritage d'Animal)

```java

class Oiseau extends Animal {
    public Oiseau(String nom) { super(nom); }

    @Override
    public void faireDuBruit() { System.out.println(getNom() + " chante."); }

    public void voler() { System.out.println("L'oiseau vole"); }

}
```

Figure 3 — Classe Oiseau (héritage + méthode spécifique voler())

Question 3 — Abstraction

L'abstraction consiste à définir CE QUE fait une méthode sans préciser COMMENT elle le fait. En Java, une classe abstraite (mot-clé abstract) ne peut pas être instanciée directement ; elle impose un contrat aux sous-classes.

Dans l'exercice :

- Animal est déclarée abstract → new Animal(...) est impossible.
- faireDuBruit() est déclarée abstract → chaque sous-classe concrète DOIT fournir son implémentation.
- Ce contrat garantit qu'on peut toujours appeler faireDuBruit() sur n'importe quel Animal, quelle que soit sa nature réelle.

Question 4 — Polymorphisme

Le polymorphisme permet à une même référence de type parent de pointer vers des objets de types enfants différents. La méthode appelée est déterminée à l'exécution (dynamic dispatch / liaison dynamique).

Dans la méthode main() :

- Animal tigre = new Mammifere("Tigre") → référence Animal, objet Mammifere.
- Animal perroquet = new Oiseau("Perroquet") → référence Animal, objet Oiseau.
- tigre.faireDuBruit() → résolu à l'exécution → "Tigre grogne."

- perroquet.faireDuBruit() → résolu à l'exécution → "Perroquet chante."

La méthode appelée est différente selon le type réel de l'objet, non selon le type de la référence.

Partie 2 : Compléter le code

Question 5 — Méthode voler() dans Oiseau

La méthode voler() est propre aux oiseaux uniquement. Elle est donc définie exclusivement dans la classe Oiseau et non dans Animal, ce qui respecte le Principe de Ségrégation des Interfaces (SOLID) : on ne force pas les mammifères à implémenter un comportement qu'ils n'ont pas.

```java

class Oiseau extends Animal {
    public Oiseau(String nom) { super(nom); }

    @Override
    public void faireDuBruit() { System.out.println(getNom() + " chante."); }

    public void voler() { System.out.println("L'oiseau vole"); }

}
```

Figure 4 — Méthode voler() complétée dans la classe Oiseau

Question 6 — Appel de voler() dans main()

Puisque voler() n'est pas déclarée dans Animal, la référence doit être de type Oiseau pour y accéder. Deux approches sont possibles :

- Option A (recommandée) : déclarer la variable directement comme Oiseau.
- Option B : transtypage (cast) explicite si on veut garder la référence de type Animal.

```java
public class Zoo {
    public static void main(String[] args) {
        Animal tigre = new Mammifere("Tigre");
        Animal perroquet = new Oiseau("Perroquet");

        tigre.faireDuBruit();     // Tigre grogne.
        perroquet.faireDuBruit(); // Perroquet chante.

        // Cast vers Oiseau pour accéder à voler() (méthode spécifique aux oiseaux)
        ((Oiseau) perroquet).voler(); // L'oiseau vole.
    }
}
```

Figure 5 — Classe Zoo complétée avec polymorphisme et appel de voler()

Partie 3 : Identification des mots-clés

Mot-clé	Utilisation dans le code
`extends`	Héritage — Mammifere extends Animal, Oiseau extends Animal
`@Override`	Redéfinition de méthode — faireDuBruit() dans Mammifere et Oiseau
`abstract`	Abstraction — classe Animal et méthode faireDuBruit()
`super(...)`	Appel au constructeur parent — dans les constructeurs de Mammifere et Oiseau
`private`	Encapsulation — attribut nom dans Animal

Exercice 1.2 – Gestion des véhicules

Partie 1 : Théorie et concepts

Question 1 — Encapsulation

Dans la classe Vehicule, les trois attributs marque, modele et annee sont déclarés private. Ils sont inaccessibles directement depuis Voiture, Camion ou toute autre classe externe. L'accès se fait exclusivement via les getters publics getMarque(), getModele(), getAnnee(). Les sous-classes elles-mêmes passent obligatoirement par ces getters (car les attributs sont private, non protected).

```java
abstract class Vehicule {
    private String marque;
    private String modele;
    private int annee;

    public Vehicule(String marque, String modele, int annee) {
        this.marque = marque;
        this.modele = modele;
        this.annee = annee;
    }

    public String getMarque() { return marque; }
    public String getModele() { return modele; }
    public int getAnnee()     { return annee; }

    public void demarrer() { System.out.println("Le véhicule démarre."); }

    public abstract void afficherDetails();
}
```

Figure 6 — Classe abstraite Vehicule avec encapsulation et méthode demarrer()

Question 2 — Héritage

L'héritage est implémenté de la façon suivante :

- Voiture extends Vehicule : hérite de marque, modele, annee et des getters. Ajoute l'attribut nombreDePortes.
- Camion extends Vehicule : hérite des mêmes membres. Ajoute l'attribut capaciteDeCharge.
- Chaque sous-classe appelle super(marque, modele, annee) pour initialiser les attributs hérités, car ils sont private dans Vehicule.

Question 3 — Polymorphisme

Dans main() :

- Vehicule maVoiture = new Voiture(...) → référence Vehicule, objet concret Voiture.
- Vehicule monCamion = new Camion(...) → référence Vehicule, objet concret Camion.
- maVoiture.afficherDetails() invoque Voiture.afficherDetails() à l'exécution.

- monCamion.afficherDetails() invoque Camion.afficherDetails() à l'exécution.

Même appel, comportements différents selon le type réel de l'objet — c'est le polymorphisme par substitution (Liskov Substitution Principle).

Question 4 — Abstraction

La classe Vehicule est abstraite car un "véhicule générique" n'a pas d'existence métier concrète. On ne veut pas écrire new Vehicule(...). La méthode afficherDetails() est abstraite car son contenu dépend entièrement du type concret (voiture affiche les portes, camion affiche la capacité). L'abstraction ici définit le contrat : tout Vehicule doit pouvoir afficher ses détails.

Partie 2 : Compléter le code

Question 5 — Méthode demarrer() et modification des sous-classes

La méthode demarrer() est ajoutée comme méthode concrète dans Vehicule. Elle affiche "Le véhicule démarre." Les sous-classes l'appellent au début de leur afficherDetails() via un simple appel à demarrer().

```java
class Voiture extends Vehicule {
    private int nombreDePortes;

    public Voiture(String marque, String modele, int annee, int nombreDePortes) {
        super(marque, modele, annee);
        this.nombreDePortes = nombreDePortes;
    }

    @Override
    public void afficherDetails() {
        demarrer();
        System.out.println("Voiture: " + getMarque() + " " + getModele()
                + " (" + getAnnee() + "), Portes: " + nombreDePortes);
    }
}
```

Figure 7 — Classe Voiture avec appel à demarrer()

```java
class Camion extends Vehicule {
    private double capaciteDeCharge;

    public Camion(String marque, String modele, int annee, double capaciteDeCharge) {
        super(marque, modele, annee);
        this.capaciteDeCharge = capaciteDeCharge;
    }

    @Override
    public void afficherDetails() {
        demarrer();
        System.out.println("Camion: " + getMarque() + " " + getModele()
                + " (" + getAnnee() + "), Capacité: " + capaciteDeCharge + " tonnes");
    }
}
```

Figure 8 — Classe Camion avec appel à demarrer()

Question 6 — Nouvelle classe Moto

La classe Moto hérite de Vehicule et ajoute un attribut typeDeGuidon. Elle implémente obligatoirement afficherDetails() (contrat imposé par la classe abstraite).

```java
class Moto extends Vehicule {
    private String typeDeGuidon;

    public Moto(String marque, String modele, int annee, String typeDeGuidon) {
        super(marque, modele, annee);
        this.typeDeGuidon = typeDeGuidon;
    }

    @Override
    public void afficherDetails() {
        demarrer();
        System.out.println("Moto: " + getMarque() + " " + getModele()
                + " (" + getAnnee() + "), Guidon: " + typeDeGuidon);
    }
}
```

Figure 9 — Nouvelle classe Moto héritant de Vehicule

```java
public class GestionTransport {
    public static void main(String[] args) {
        Vehicule maVoiture = new Voiture("Toyota", "Corolla", 2021, 4);
        Vehicule monCamion = new Camion("Volvo", "FMX", 2019, 12.5);
        Vehicule maMoto    = new Moto("Honda", "CB500", 2022, "Guidon sport");

        maVoiture.afficherDetails();
        System.out.println();
        monCamion.afficherDetails();
        System.out.println();
        maMoto.afficherDetails();
    }
}
```

Figure 10 — Classe principale GestionTransport avec polymorphisme et boucle sur tableau Vehicule[]

Partie 3 : Identification des mots-clés

Mot-clé	Utilisation dans le code
`@Override`	Redéfinition de méthode — afficherDetails() dans Voiture, Camion, Moto
`extends`	Héritage — Voiture extends Vehicule, Camion extends Vehicule, Moto extends Vehicule
`super(...)`	Appel au constructeur parent — super(marque, modele, annee) dans chaque sous-classe
`abstract`	Abstraction — classe Vehicule et méthode afficherDetails()
`private`	Encapsulation — attributs marque, modele, annee dans Vehicule

Exercice 1.3 – Gestion des comptes bancaires

Implémentation complète d'un système bancaire à partir de zéro, en appliquant tous les concepts POO vus dans les exercices précédents.

Architecture du système

Le système est composé de 4 classes :

- CompteBancaire : classe de base (non abstraite) avec dépôt et retrait.
- CompteCourant : hérite de CompteBancaire, ajoute la gestion du découvert autorisé.
- CompteEpargne : hérite de CompteBancaire, ajoute le calcul des intérêts.
- Banque : classe principale avec main(), démonstration de toutes les fonctionnalités.

Implémentation

Classe CompteBancaire

La classe de base encapsule le numéro de compte (final, immuable) et le solde. Elle inclut une validation des montants pour toutes les opérations, et expose un setSolde() protégé pour les sous-classes.

```java
class CompteBancaire {  no usages
    protected String numeroCompte;  5 usages
    protected double solde;  8 usages

    public CompteBancaire(String numeroCompte, double soldeInitial) {  no usages
        this.numeroCompte = numeroCompte;
        this.solde = soldeInitial;
    }
    public void deposer(double montant) {  no usages
        if (montant > 0) {
            solde += montant;
            System.out.println("Dépôt de " + montant + " € sur le compte " + numeroCompte
                    + ". Nouveau solde : " + solde + " €");
        }
    }
    public void retirer(double montant) {  no usages
        if (montant > 0 && montant <= solde) {
            solde -= montant;
            System.out.println("Retrait de " + montant + " € du compte " + numeroCompte
                    + ". Nouveau solde : " + solde + " €");
        } else {
            System.out.println("Solde insuffisant sur le compte " + numeroCompte
                    + ". Solde actuel : " + solde + " €");
        }
    }
    public double getSolde() { return solde; }  no usages
    public String getNumeroCompte() { return numeroCompte; }  no usages
}
```

Figure 11 — Classe de base CompteBancaire avec encapsulation et validation

Classe CompteCourant

CompteCourant surcharge la méthode retirer() pour autoriser un solde négatif dans la limite du découvert autorisé. C'est un exemple de redéfinition de comportement par héritage.

```
class CompteCourant extends CompteBancaire {  no usages

    public CompteCourant(String numeroCompte, double soldeInitial) {  no usages

        super(numeroCompte, soldeInitial);
    }
}
```

Figure 12 — Classe CompteCourant avec gestion du découvert

Classe CompteEpargne

CompteEpargne ajoute la méthode calculerInterets(double taux) qui applique un taux annuel en pourcentage au solde courant. Elle utilise setSolde() protégé hérité de CompteBancaire.

```
1   class CompteEpargne extends CompteBancaire {
2
3       public CompteEpargne(String numeroCompte, double soldeInitial) { super(numeroCompte, soldeInitial); }
6
7       public void calculerInterets(double taux) {
8           double interets = solde * taux / 100;
9           solde += interets;
10          System.out.println("Intérêts calculés (" + taux + "%) : +" + interets
11                  + " €. Nouveau solde : " + solde + " €");
12      }
13  }
```

Figure 13 — Classe CompteEpargne avec calcul d'intérêts

Classe principale Banque

La classe Banque démontre l'ensemble des fonctionnalités : dépôts, retraits valides, retraits refusés, calcul d'intérêts, et polymorphisme via un tableau CompteBancaire[].

```java
public class Banque {
    public static void main(String[] args) {
        CompteCourant cc = new CompteCourant("CC-001", 1000.0);
        CompteEpargne ce = new CompteEpargne("CE-001", 5000.0);

        System.out.println("=== Compte Courant ===");
        cc.deposer(500);
        cc.retirer(200);
        cc.retirer(2000); // solde insuffisant

        System.out.println("\n=== Compte Epargne ===");
        ce.deposer(1000);
        ce.calculerInterets(3.5);
        ce.retirer(100);
    }
}
```

Figure 14 — Classe principale Banque avec tests complets

Récapitulatif des concepts POO appliqués

Mot-clé	Utilisation dans le code
`private`	Encapsulation — numeroCompte et solde dans CompteBancaire
`protected`	Accès contrôlé — setSolde() accessible aux sous-classes uniquement
`extends`	Héritage — CompteCourant et CompteEpargne héritent de CompteBancaire
`@Override`	Redéfinition — retirer() dans CompteCourant, afficherInfos() dans les deux sous-classes
`super(...)`	Appel constructeur parent — dans CompteCourant et CompteEpargne

Conclusion

Ce TP a permis de mettre en pratique les quatre piliers fondamentaux de la Programmation Orientée Objet en Java :

- Encapsulation : protection des attributs via private et accès contrôlé via getters/setters.
- Héritage : réutilisation et spécialisation du code via extends et super().

- Abstraction : définition de contrats via abstract, interdisant l'instanciation directe des classes génériques.
- Polymorphisme : traitement uniforme d'objets de types différents via des références de type parent, avec résolution dynamique des méthodes à l'exécution.

Les trois exercices ont progressivement illustré ces concepts : du Zoo (introduction) au système de transport (consolidation) en passant par la banque (implémentation complète depuis zéro). L'utilisation de principes SOLID — notamment la ségrégation des interfaces et la responsabilité unique — a guidé la conception de chaque classe.